



西北工业大学

NSCSCC2026— 第十届“龙芯杯”

西北工业大学六队决赛提交作品报告

团队成员： 杨思哲、 钟佳诚

指导教师： 王党辉、 王继禾

日 期： 2026 年 8 月 16 日

目录

第一章 概述	3
1.1 设计简介	3
1.2 超标量处理器架构总览	3
第二章 分支预测与存储子系统设计	6
2.1 分支预测	6
2.2 地址转换	6
2.3 高速缓存	8
第三章 处理器后端实现	13
3.1 动态调度设计	13
3.2 访存指令的执行级实现	14
3.3 提交级设计	15
第四章 结果展示与参考设计说明	17
4.1 启动 Linux 系统	17
4.2 参考设计说明	17
参考文献	18

第一章 概述

1.1 设计简介

本设计是一个基于 LoongArch32 位精简版指令集^[1]，采用乱序双发射十级流水线架构，频率可达 85Mhz 的处理器软核。本设计能够通过比赛初赛提供的功能测试的 58 个测试点，性能测试分数为 2.39，并正常启动 Linux 操作系统^[2]。

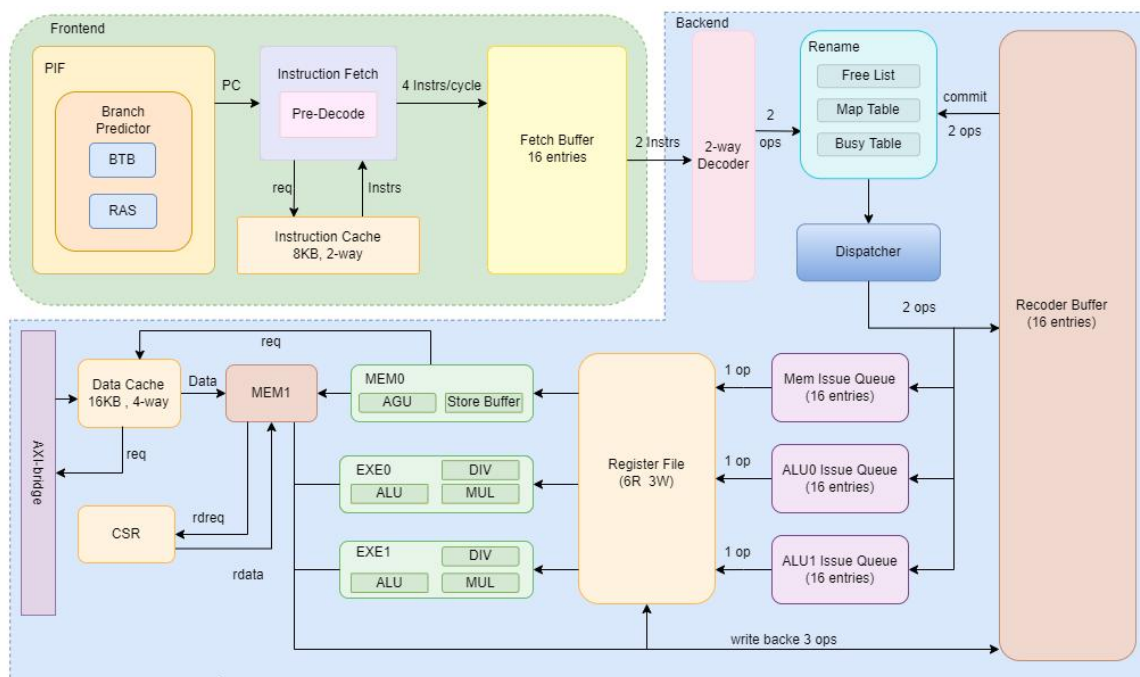
设计集成了 AXI 总线、Cache 和内存管理单元(MMU)，并实现了分支预测。处理器包含的指令 Cache 为两路组相联设计，每路 256 项，一项 128 位，共 8KiB。处理器包含的数据 Cache 为四路组相连设计，每路 128 项，一项 256 位，共 16KiB。

TLB 为二级 TLB 设计，第一级 ITLB 和 DTLB 均使用 4 项直接映射结构，第二级混合 TLB 使用 32 项全相联结构，支持 4K 页和 4M 页。

本设计实现了除浮点外全部整数、特权以及 cpucfg 指令，支持例外与中断机制，实现了除浮点相关外全部例外，设计了 CRMD 等 36 项控制状态寄存器。

1.2 超标量处理器架构总览

本课题设计并实现了双发射乱序执行超标量处理器。如图 1-1 所示，该架构在逻辑上被严格划分为前端（Frontend）与后端（Backend）两大解耦域，整体数据流严格遵循“顺序取指译码、乱序发射执行、绝对顺序提交”的经典动态调度范式。



处理器采取前后端设计,前端流水级数为 3,后端共 7 个流水级,共 10 个流水级,最深级数为 11。在数据流的起始端,处理器前端的核心任务是为后端持续输送指令流。预取指(PIF)阶段集成了由分支目标缓冲(BTB)和返回地址栈(RAS)构成的复合分支预测器,负责在指令缓存命中前提供前瞻性的程序计数器。本架构的指令缓存配置为 8KB 容量与两路组相联结构。为了平滑存储层次的读取延迟并有效掩盖分支预测可能引入的流水线气泡,前端具备单周期抓取并预译码(Pre-Decode)最多 4 条指令的能力,并将其暂存入容量为 16 项的取指缓冲队列(Fetch Buffer)中。这种 4 条指令入队、后端 2 条指令出队的非对称带宽设计,在物理时序上实现了前后端深度的动态解耦。

跨越前后端边界后，指令从取指缓冲队列以双发射（2 Instrs/cycle）的稳定带宽流入后端双路译码器（2-way Decoder）。解析后的指令随即进入乱序执行引擎的第一个核心枢纽——寄存器重命名模块。本设计采用基于统一物理寄存器堆的重命名架构。通过空闲列表、映射表以及记录操作数就绪状态的 Busy Table 的紧密协同。其中映射表包括架构映射表（aRAT）和推测映射表（sRAT）。该模块将指令中有限的逻辑寄存器索引动态映射至更大的物理地址空间，从根本上解除了写后写与读后写引发的名称假相关阻塞。

完成重命名与解绑的微操作随后进入分派器进行路由调度。在一个时钟周期内，分派逻辑必须将 2 条微操作同步分发至两个核心控制阵列：其一是按程序原始顺序写入容量为 16 项的重排序缓存，以为后续的精确异常恢复保留确切的状态快照；其二是根据微操作的具体执行类型，将其精准路由至后端的分布式发射队列中。本架构针对不同运算类型构建了三个相互独立的 16 项发射队列，包含两个算术逻辑队列（ALU0/ALU1 Issue Queue）与一个访存队列（Mem Issue Queue）。

脱离了静态顺序束缚的指令在各自的发射队列中实时监听操作数的就绪状

态。一旦源操作数齐备，指令便被乱序唤醒，并前往具有 6 个读端口和 3 个写端口（6R 3W）的超宽物理寄存器堆提取数据。随后，指令涌入多路并行的底层功能阵列进行计算。执行阵列被划分为两条包含通用算术、乘法和除法的复合计算管道（EXE0, EXE1），以及一条包含地址生成（AGU）和存储缓冲（Store Buffer）的专属访存管道（MEM0, MEM1）。这三个执行级通道分别对应着三个发射队列，队列和执行级通道存在着对应关系。比如 ALU0 Issue Queue 里的指令不允许进入 EXE1 或 MEM0。在计算周期的末端，最多 3 条指令的结果会同步写回物理寄存器堆并更新 ROB 的完成状态标志。最终，位于 ROB 头部的指令在接受严格校验、确认无推测异常与分支误预测后，以最高 2 ops/cycle 的带宽安全退休（Commit），从而完成整个系统架构状态的更新。

第二章 分支预测与存储子系统设计

2.1 分支预测

本文设计的分支预测模块采用 BHT（Branch History Table，分支历史表）+ BTB（Branch Target Buffer，分支目标缓冲）+ RAS（Return Address Stack，返回地址栈）的混合预测结构，能够支持 LoongArch 架构下的条件分支、无条件跳转、函数调用以及函数返回等典型控制转移指令。该模块通过分支方向预测、目标地址预测和返回地址预测的协同工作，为取指单元提前提供下一条取指地址，从而降低控制相关造成的流水线停顿，提高前端连续取指能力。

分支预测模块整体结构如图 3-1 所示，其内部主要由 BHT、BTB、RAS 以及预测判决逻辑组成。其中，BHT 负责条件分支的方向预测，BTB 负责缓存分支目标地址并记录相应的分支类型信息，RAS 则用于提高函数返回类指令的目标地址预测准确性。

从整体工作流程来看，分支预测模块主要包括预测查询通路和状态更新通路两部分。在预测查询阶段，取指单元将当前取指 PC 送入分支预测模块，模块同时访问 BTB 和 BHT。BTB 根据当前 PC 判断对应的控制转移指令是否已经被记录，并在命中时给出相应的分支目标地址及分支类型信息；BHT 则根据当前 PC 索引相应表项，直接产生条件分支的方向预测结果。对于普通条件分支，预测判决逻辑综合 BHT 的方向预测结果和 BTB 的命中情况，确定当前指令是否预测跳转；对于无条件跳转和函数调用类指令，模块主要依据 BTB 中记录的分支类型和目标地址产生预测结果；对于函数返回类指令，则进一步结合 BTB 给出的分支类型信息和 RAS 状态，由 RAS 提供预测返回地址。最终，预测判决逻辑统一生成预测跳转标志 `pred_taken` 和预测目标地址 `pred_target`，供取指前端选择后续取指 PC。

在状态更新阶段，模块根据分支指令的实际执行结果对各预测结构进行修正。BTB 主要在分支指令的实际目标地址和分支类型确定后进行更新，将对应的 PC、目标地址及分支类型写入表项，以便后续取指阶段能够快速识别该控制转移指令并获得预测目标地址。BHT 则根据条件分支的真实跳转结果更新相应预测状态，使后续遇到相同或相关分支时能够根据历史执行行为调整方向预测结果。对于函数调用和函数返回指令，RAS 根据相应的控制信息完成返回地址的压栈和出栈操作。通过上述机制，分支预测模块能够根据程序运行过程中实际发生的控制流行为持续修正预测状态，提高分支方向和目标地址预测的准确性。

2.2 地址转换

为了降低虚拟地址到物理地址转换过程对处理器流水线造成的影响，本设计在存储子系统中设计了二级 TLB 结构。TLB 模块用于缓存近期使用的虚拟页号到物理页号的映射关系，使处理器在取指和访存阶段能够快速完成地址转换。相比于单级 TLB 结构，二级 TLB 能够在访问延迟和命中率之间取得较好的折中：

一级 TLB 规模较小、访问速度快，用于满足高频地址转换请求；二级 TLB 容量相对更大，用于保存更完整的地址映射信息，并在一级 TLB 缺失时提供补充查询和回填数据。本文实现的 TLB 模块由一级指令 TLB、一级数据 TLB 和二级 TLB 三部分组成，整体结构如图 2- 1 所示。顶层 tlb 模块对外提供两个独立的查询端口，其中查询端口 0 主要服务于取指侧地址转换请求，查询端口 1 主要服务于访存侧地址转换请求。两个端口均输入虚拟页号 vppn、奇偶页选择信号 odd_page、地址空间标识符 asid 以及查询有效信号 valid，并输出查询是否命中、表项索引、页大小、物理页号、页有效位、脏位、存储访问属性和特权等级等信息。

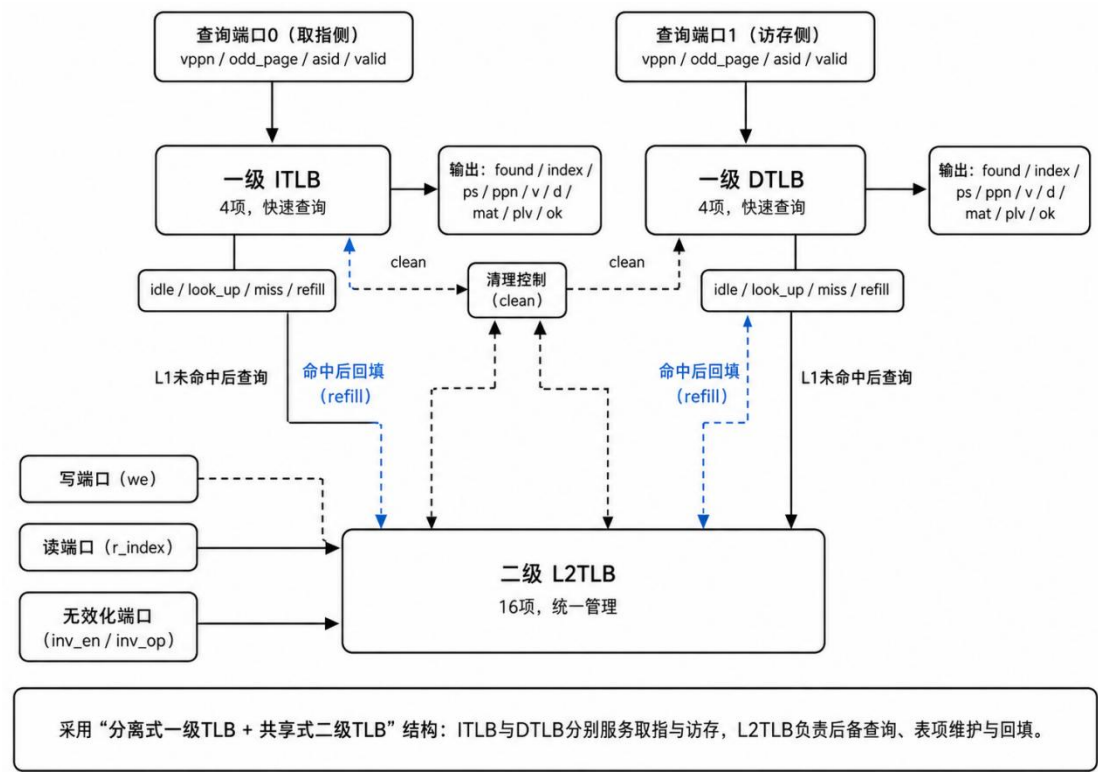


图 2- 1TLB 整体架构

一级 TLB 采用小容量快速查询结构，主要用于缓存近期访问过的地址映射。1l1tlb 内部设置 4 个表项，每个表项保存虚拟页号、ASID、全局页标志、页大小、有效位以及偶页和奇页对应的物理页号、特权等级、存储属性、脏位和页有效位等信息。一级 TLB 在查询时根据虚拟页号和 ASID 的部分位生成哈希索引，并直接访问对应表项完成匹配判断。这种设计避免了一级 TLB 中复杂的全相联比较逻辑，有利于缩短访问路径，提高取指和访存阶段的地址转换速度。查询命中条件主要包括虚拟页号匹配、ASID 匹配或全局页标志有效，以及表项有效位。

二级 TLB 用于在一级 TLB 缺失时提供进一步查询，其容量由参数 TLBNUM 决定，本文实现中默认为 16 项。与一级 TLB 的哈希直接访问方式不同，二级

TLB 采用并行匹配结构。模块内部为两个查询端口分别生成匹配向量 `match0` 和 `match1`，对所有 TLB 表项同时进行虚拟页号、ASID 或全局页标志以及有效位判断。当某一表项满足匹配条件时，编码器将匹配向量转换为命中表项索引，并输出该表项中的页大小、物理页号、权限等级、存储属性、脏位和有效位等完整信息。由于二级 TLB 容量大于一级 TLB，且采用统一结构保存取指和访存共享的地址映射，因此可以在一级 TLB 未命中时提高整体地址转换的命中率。

TLB 查询过程由顶层模块中的有限状态机进行控制。ITLB 和 DTLB 分别具有独立的状态机，状态包括 `idle`、`look_up`、`miss` 和 `refill`。当查询端口有效时，状态机由空闲状态进入查询状态，首先判断一级 TLB 是否命中；若一级 TLB 命中，则直接向流水线返回地址转换结果，并通过 `s0_ok` 或 `s1_ok` 表示查询完成。若一级 TLB 未命中，则继续根据二级 TLB 的查询结果决定后续处理：当二级 TLB 命中时，状态机进入 `refill` 状态，将二级 TLB 中的表项信息回填到对应的一级 TLB 中，使后续相同地址访问能够在一级 TLB 中快速命中；当二级 TLB 也未命中时，状态机进入 `miss` 状态，表示当前地址转换无法由 TLB 完成，需要后续异常处理或页表管理机制进一步处理。

在 TLB 维护机制方面，二级 TLB 提供写端口、读端口和无效化端口。写端口用于向指定索引写入新的 TLB 表项，写入内容包括虚拟页号、ASID、全局页标志、页大小、表项有效位以及两个页对应的物理页号和属性信息；读端口则根据索引读取二级 TLB 中的表项内容，便于系统控制指令访问 TLB 状态。无效化端口通过 `inv_en` 和 `inv_op` 实现不同范围的 TLB 表项失效操作，包括全部失效、全局页失效、非全局页失效、指定 ASID 失效以及指定 ASID 和虚拟页号失效等情况。为了保证一级 TLB 与二级 TLB 内容一致，当二级 TLB 发生写入或执行无效化操作时，顶层模块会通过 `clean` 信号清空一级 ITLB 和 DTLB 中的有效位，避免一级 TLB 继续命中过期表项，从而保证地址转换结果的正确性。

2.3 高速缓存

2.3.1 Cache 整体架构

Cache 采用阻塞式数据 Cache 结构，即同一时刻仅处理一个缺失请求，在缺失处理期间暂停新的缺失访问。同时通过写缓冲减少写命中操作对流水线的阻塞。该设计在硬件资源、控制复杂度和性能提升之间取得折中，适合作为中小规模乱序处理器的数据 Cache 实现方案。Cache 整体结构可概括为图 2-2 所示。CPU 侧接口用于接收访存请求，包括读写类型、访问大小、地址索引、标记、块内偏移、写字节使能和写数据等信息。Cache 内部通过 `TagV` 阵列判断命中，通过 `Data`

阵列保存缓存数据，通过 Dirty 寄存器记录 Cache 行是否被修改。AXI 侧接口则负责在 Cache 缺失或非缓存访问时与下级存储系统进行数据交互。此外，模块还支持 Cache 维护指令和异常取消信号，使 Cache 能够与异常处理和特权指令执行过程协同工作。

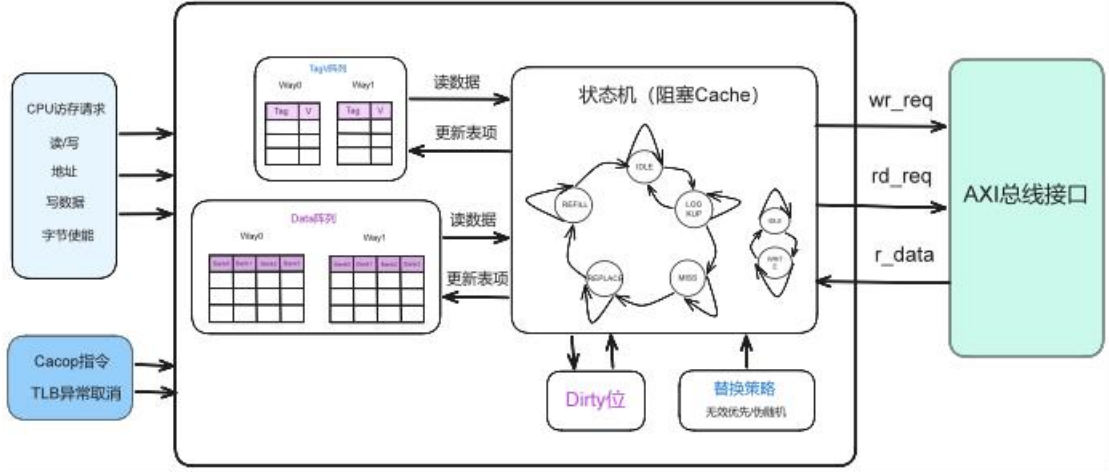


图 2-2 Cache 整体架构

2.3.2 VIPT 访问方式与地址划分

本文设计的指令 Cache 采用两路组相联结构，数据 Cache 采用四路组相连，Cache 行大小为 16B，采用“虚 Index 实 Tag”（VIPT）的访问形式。在 VIPT 访问方式下，Cache 查找阶段可利用虚拟地址的 Index 提前访问 Data 阵列和 TagV 阵列，而 Tag 比较则使用 TLB 转换后的物理 Tag 完成。该方式能够使 Cache 阵列访问与地址转换过程部分重叠，从而缩短访问路径。然而，VIPT 结构需要避免不同虚拟地址映射到同一物理地址时落入不同 Cache 组的问题，即 Cache 别名问题。为此，Cache 每一路容量通常不应超过页大小。由于 Index 与 Offset 均落在页内偏移范围内，虚拟地址和物理地址在该部分保持一致，因此不同虚拟地址只要映射到同一物理页，其 Cache 组索引保持一致，从结构上避免了 VIPT 别名问题。因此，本设计将每一路 Cache 容量定义为 4KB，使得 ICache 总容量达到 8KB，DCache 总容量达到 16KB。

Cache 地址被划分为 Tag、Index 和 Offset 三部分，Index 用于定位 Cache 中的具体组，Tag 用于与该组内各路保存的标记进行比较，Offset 用于选择 Cache 行内的目标数据。结合上述设计规格，可计算出地址各部分的位数：物理地址 [31:12] 作为 Tag，[11:4] 作为 Index，[3:0] 作为 Offset。访存请求进入 Cache 后，模块首先根据 Index 同时读取两路 TagV 和 Data 内容，并将请求 Tag 与两路 Tag 进行比较：若某一路有效位为 1 且 Tag 匹配，则判定该请求命中；若两路均未命中，则进入缺失处理流程。上述整个 Cache 访问流程的具体逻辑，可参考图 2-3 的详细示意。

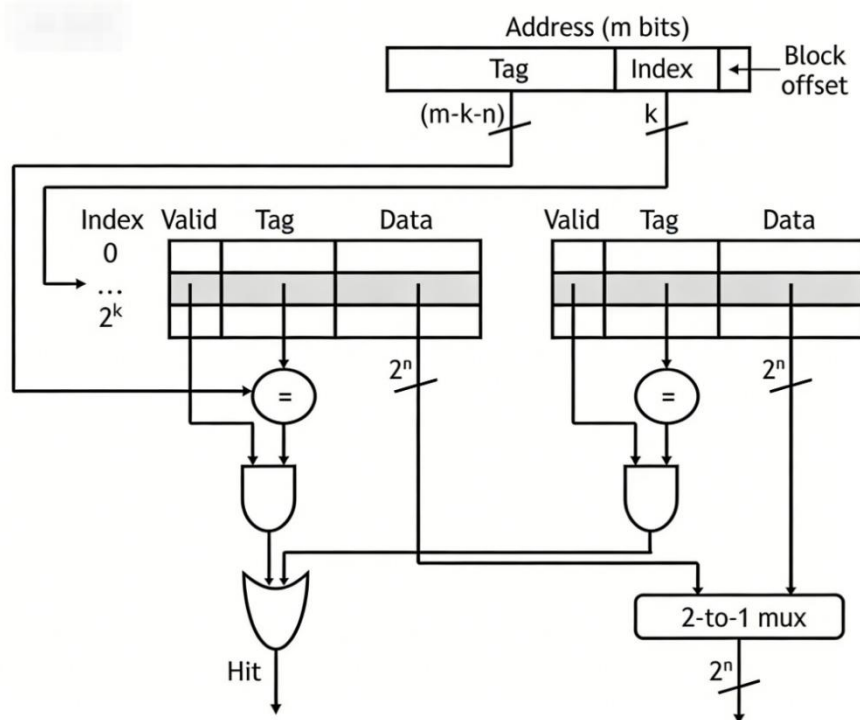


图 2-3 两路组相联 Cache 访问示意图

2.3.3 Cache 缺失处理与替换策略

如图 2-4 所示，当 Cache 访问未命中时，模块首先根据替换策略选择待替换路。若选中的 Cache 行无效，则可直接使用该行进行缺失填充。若该行有效但 Dirty 位为 0，则说明该行内容与下级存储一致，也可直接替换。若该行有效且 Dirty 位为 1，则必须先将该 Cache 行写回下级存储系统，避免修改后的数据丢失。写回完成后，Cache 再向 AXI 发起读请求，将目标地址所在 Cache 行重新读入 Data 阵列，同时更新 TagV 并清除 Dirty 位。对于 Load 缺失，若返回数据中已经包含 CPU 当前需要的字或字节，则可提前向 CPU 置位 `data_ok`；对于 Store 缺失，则需要在 Refill 完成后将写数据合并到新填充的 Cache 行中，并置位 Dirty 位。

在替换策略方面，本文采用无效行优先与 LFSR 随机替换相结合的方式。当某一组中存在无效 Cache 行时，优先选择无效行作为替换目标，从而避免不必要的有效数据淘汰；当两路均有效时，利用 LFSR 产生的伪随机结果选择替换路。与 LRU 等精确替换策略相比，LFSR 随机替换硬件实现简单，不需要为每组维护复杂的访问顺序状态，能够降低面积和时序开销。

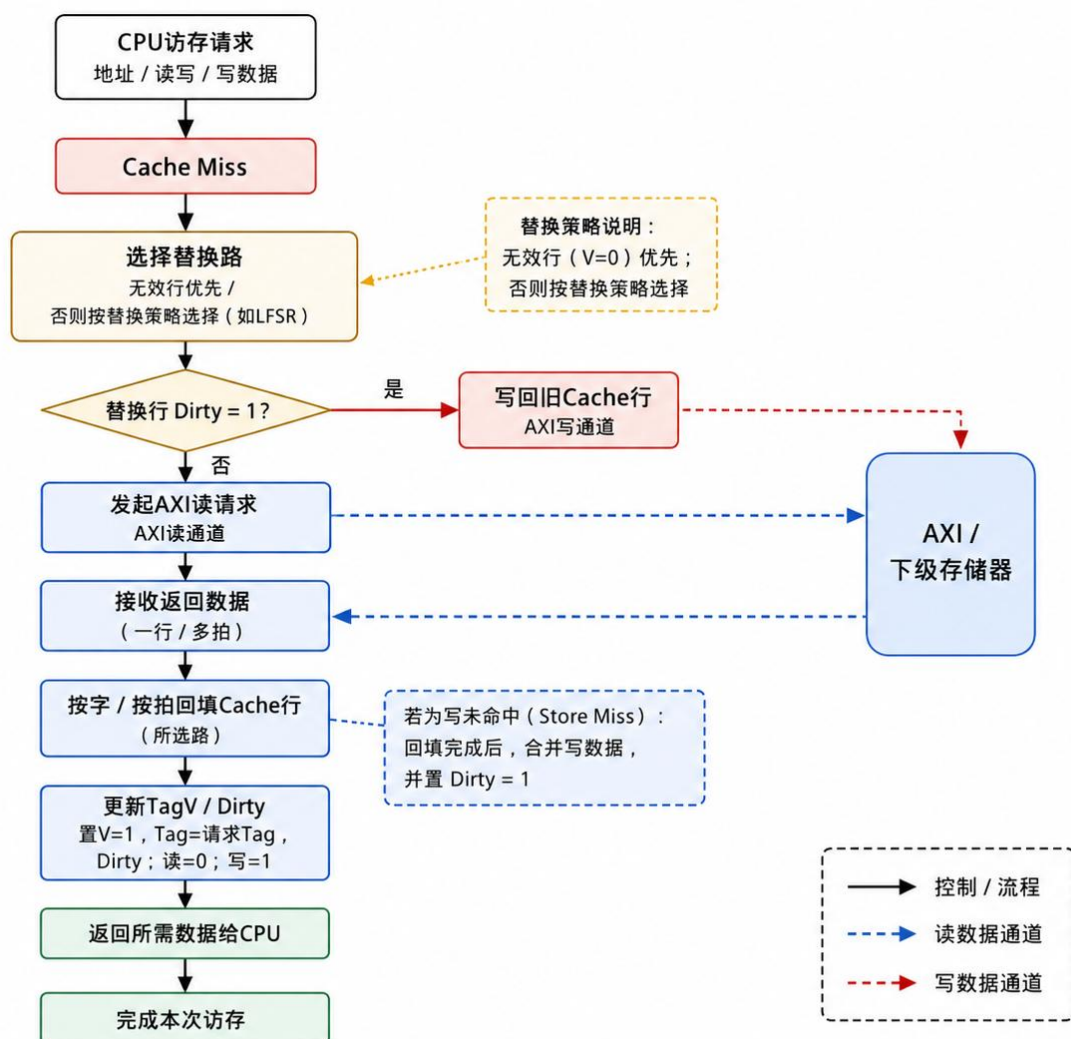


图2- 4 Cache缺失处理

2.3.4 主状态机设计

为了控制 Cache 访问过程，本文设计了一个主状态机，其状态包括 idle、lookup、miss、replace 和 refill 五个阶段。图 2- 5 展示了 Cache 主状态机的状态转换。在 IDLE 状态下，如果 CPU 发出有效请求，且当前写缓冲不会与该请求发生 Bank 冲突，则 Cache 接收请求并进入 LOOKUP 状态。在 LOOKUP 状态下，若请求命中，则读请求直接返回数据，写请求进入写缓冲，然后状态机回到 IDLE；若请求未命中，则根据替换行的 Dirty 位和有效位判断是否需要进入 MISS 状态执行写回。如果不需要写回，则直接进入 REPLACE 状态发起读请求。MISS 状态用于处理脏块写回。当 AXI 写通道准备好后，Cache 向下级存储系统提交写请求，并在写请求被接收后进入 REPLACE 状态。REPLACE 状态用于发起读请求，当 AXI 读通道准备好后进入 REFILL 状态。

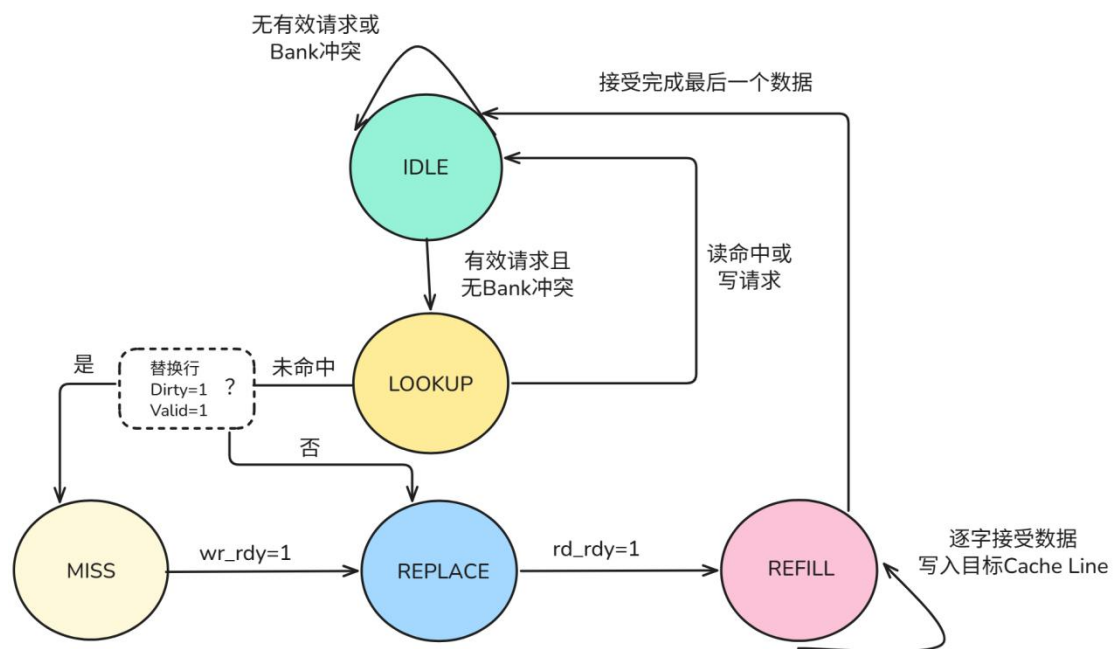


图 2- 5 Cache 主状态机转换

第三章 处理器后端实现

3.1 动态调度设计

本设计采用了分布式、非压缩式的发射队列架构。以图 3- 1 ALU0 Issue Queue 设计所示的 ALU0 发射队列被配置为每周期单入单出、深度为 16 项的独立缓存池，专门负责为 EXE0 执行通道提供源源不断的就绪指令。

在指令的入队与分配阶段，发射队列通过前端数据总线（m_to_iq_bus）接收来自紧耦合分发级的微操作。队列在每个时钟周期通过纯组合逻辑从索引 0 开始对整个 16 项寄存器阵列进行查询，捕捉有效位为低电平的空闲表项，并将新指令的年龄标签及操作数就绪状态等控制信息写入该槽位。同时，队列的顶层控制逻辑会实时评估当前的空闲槽位数量。一旦检测到队列即将满载，便立刻撤销握手许可信号（iq_allowin）。当指令安全驻留在发射队列后，便进入了对其源操作数状态进行唤醒阶段。运算指令进入计算通道时，会通过旁路提前更新队列的操作数状态，从而实现大部分指令的背靠背运行。在发射队列内部的更新操作数状态。本设计依托常规的寄存器阵列，在时序逻辑块中通过非阻塞赋值机制构建了并行比较网络。在每一个时钟周期，硬件通过硬连线展开的循环遍历逻辑，将广播来的目标寄存器编号与队列中 16 个表项的源操作数编号进行并发比对。一旦检测到物理地址一致，硬件会在当前节拍的末尾将对应表项的操作数就绪位置为高电平。运算指令在执行级执行完毕后，会将结果进行全局广播更新就绪状态表和重排序缓存。

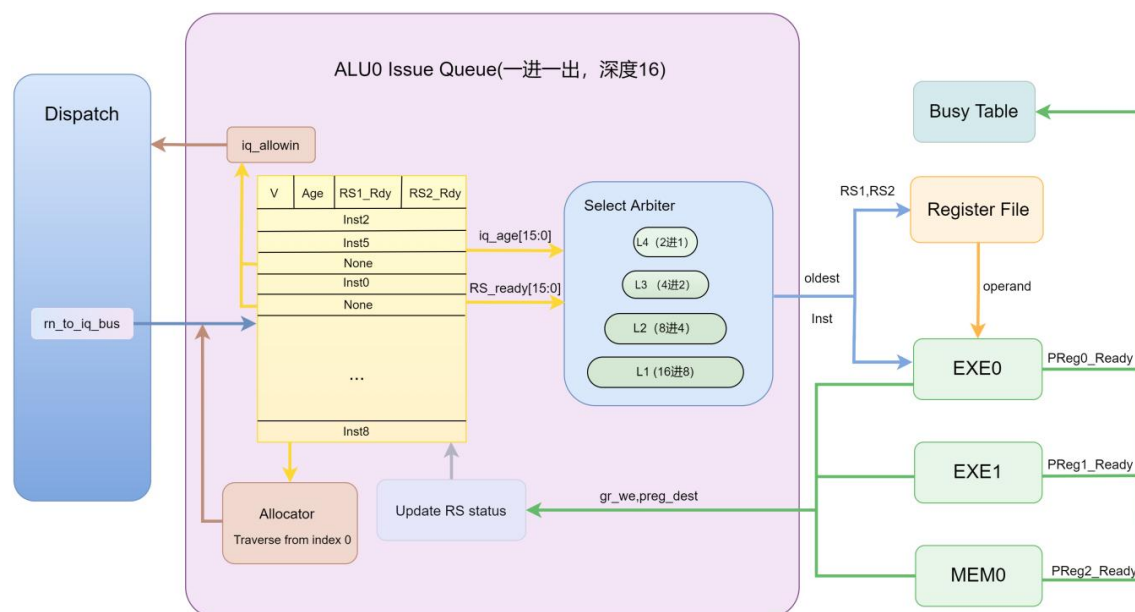


图 3- 1 ALU0 Issue Queue 设计

随着多条指令在同一周期内被唤醒，发射队列随即进入指令挑选阶段。当队列中存在多条指令的源操作数均已齐备时，挑选仲裁器（Select Arbiter）必须在一个极短的时钟周期内决出唯一胜者。为了降低指令由于相关性而无法发射的概率，本微架构设计了一棵极其精密的高速锦标赛仲裁树。该仲裁树共分为四级物

理流水级（L1 至 L4），分别执行 16 进 8、8 进 4、4 进 2 以及 2 进 1 的两两淘汰对决。每条指令在入队的时候年龄都为 0，当每周期有新的指令入队的时候，所有已经在队列的旧指令的年龄都会递增。在每一级的比较节点上，仲裁逻辑严格提取指令的年龄位（iq_age[15:0]）作为胜负判据。确保在所有就绪指令中，程序原始顺序中最老的那条指令能够过关斩将直达根节点。最终胜出的指令将被正式弹出队列，其携带的物理地址标签将直接驱动物理寄存器堆进行数据预取。随后将指令送入 EXE0 执行单元。（后面为提升主频删除了最旧指令优先发射和旁路前递）

3.2 访存指令的执行级实现

在双发射乱序处理器的微架构中，访存执行通道负责连接处理器执行核心与存储层次，是 Load、Store 指令完成地址生成、地址转换、Cache 访问以及数据返回的主要通路。本设计将访存指令的执行过程划分为地址计算与访存请求级（MEM0）和数据返回与写回级（MEM）两个流水阶段，并在访存通路中集成 MMU、TLB、DCache 和 Store Buffer，以完成虚实地址转换、缓存访问以及推测状态下的存储管理。

访存指令进入 MEM0 级后，首先根据基址寄存器和指令中的立即数偏移计算有效地址，该地址作为访存操作的虚拟地址。与此同时，MEM0 级根据访存数据宽度进行地址对齐检查。当检测到非对齐访问时，处理器产生相应的地址异常，并将异常信息随指令继续向后传递，同时抑制后续的实际存储访问请求，以避免异常指令对存储系统状态产生影响。

对于通过地址对齐检查的访存请求，虚拟地址随后送入 MMU。MMU 根据当前地址转换模式完成虚拟地址到物理地址的映射，其中需要地址转换的访问通过 TLB 查询获得对应的物理页号，并进行有效性和访问权限检查。若地址转换过程中发生 TLB 未命中、页无效或访问权限不满足等情况，则由 MMU 产生相应异常信息，并取消本次实际 Cache 访问；若地址转换正常完成，则得到最终的物理访存地址，并将其送往后续的 Store Buffer 和 DCache 访问通路。

对于 Load 指令，处理器在获得物理地址后首先利用该地址查询 Store Buffer，以检查是否存在尚未提交但地址重叠的更早 Store 指令。当 Store Buffer 中存在可前递的数据时，根据字节写掩码生成相应的前递数据和有效字节信息；对于无法完全由 Store Buffer 提供的数据，则继续访问 DCache。DCache 根据物理地址完成组索引和 Tag 匹配，在 Cache 命中时直接返回对应的数据；在未命中时则通过下层存储接口完成数据块获取和 Cache 行填充。进入 MEM 级后，来自 DCache 的读数据与 Store Buffer 的前递数据按照有效字节进行合并，随后根据 Load 指令的数据宽度及符号扩展方式生成最终结果，并通过写回网络写入对应的物理寄存器。

在整个访存微架构中，Store Buffer 是实现乱序执行和精确状态恢复的重要组成部分。如果推测执行阶段的 Store 指令直接修改 DCache，那么一旦后续发生分支预测失败或异常，就需要撤销已经产生的存储副作用，硬件实现较为复杂。因此，本设计不允许尚未提交的 Store 指令直接修改 Cache，而是利用 Store Buffer 暂存其访存信息。

当 Store 指令在 MEM0 级完成地址计算和地址转换后，其物理地址、待写数据以及字节写使能等信息被写入 Store Buffer。此时 Store 指令在执行通路中

的相关操作已经完成，但其数据仍处于推测状态，不会立即写入 DCache。Store Buffer 因而在乱序执行核心与具有架构可见副作用的存储系统之间建立了隔离，使处理器可以继续执行后续指令，而无需等待实际的 Cache 写入完成。

当 Store 指令最终到达重排序缓存（ROB）头部并满足提交条件后，对应的 Store Buffer 表项才获得提交许可。Store Buffer 按照程序顺序将已经确认提交的写请求发送至 DCache，由 DCache 完成后续的命中判断、数据更新以及必要的缺失处理。若在 Store 指令正式提交之前发生异常或流水线冲刷，则尚处于推测状态的 Store Buffer 表项可以被直接取消，而不会对 DCache 和下层存储系统造成错误修改，从而保证处理器能够实现精确异常和正确的状态恢复。

通过上述设计，MEM0、MMU、Store Buffer、DCache 与 MEM 级共同构成完整的访存执行通路。MEM0 负责地址生成和访问属性判断，MMU 负责虚实地址转换及相关异常检查，Store Buffer 负责未提交 Store 的暂存以及 Store-to-Load 数据前递，DCache 负责实际的数据缓存访问，而 MEM 级则完成返回数据处理和结果写回。各模块相互配合，在保证精确异常和程序语义正确性的同时，降低了访存延迟对乱序流水线执行效率的影响。

3.3 提交级设计

在乱序超标量架构中，提交级的任务是把底层混沌的乱序执行结果重新梳理成符合顺序语义的精确架构状态。

如图 3-2 所示，在每个时钟周期的上升沿，提交级探测模块会优先检查位于重排序缓存头指针处指令的完成标志。若该首发指令尚未算出结果，说明程序仍处于物理执行的挂起状态。硬件会直接将提交弹出计数器强置为零，彻底封锁整条提交通路，严禁任何后续乱序完成的推测指令越权修改系统状态。如果首发指令就绪，硬件将根据其携带的控制信标触发多级联动的判决逻辑。首先面临的是优先级的异常与中断判定。若发现是常规异常或异常返回指令，流水线将瞬间拉高全局冲刷信号。并把异常向量与正确的恢复地址广播至全系统，同时截断该指令及其后续所有错误推测。若该指令为跳转分支，提交级会激活分支预测器进行评估。一旦判定发生预测失败，流水线不仅会拉高冲刷信号，还会利用正确的目标地址引导取指单元进行精确跳转。此外，诸如控制与状态寄存器写操作、TLB 读写或高速缓存维护等特权指令，由于极易引发全局数据不一致，被硬件设计为非推测性的序列化点。执行此类指令后，微架构同样会触发强制冲刷操作。将取指程序计数器定向至下一条指令，确保系统在无推测污染的干净状态下继续执行。在排除了上述所有异常和复杂行为后，若首发指令被判定为常规安全指令，硬件会将其弹出计数置为一。紧接着探查并行的第二条指令。如果第二条指令也已就绪并且安全，弹出计数即可升为二，从而实现并行引退。

当指令被判定为可安全提交时，流水线必须进行物理资源的释放和映射关系的固化。提交逻辑会提取指令绑定的架构寄存器号及物理寄存器号，向架构映射表通报其最新绑定的物理寄存器。这样一来这一推测映射就会转正为永久架构状态。指令覆盖的旧物理寄存器会返还给空闲列表，为后续的重命名分配补足硬件资源。对于写入内存的操作，提交级会利用专门的提交计数器，精准通知底层的写缓冲阵列将推测态的 store 指令转为架构态，等待访问内存。

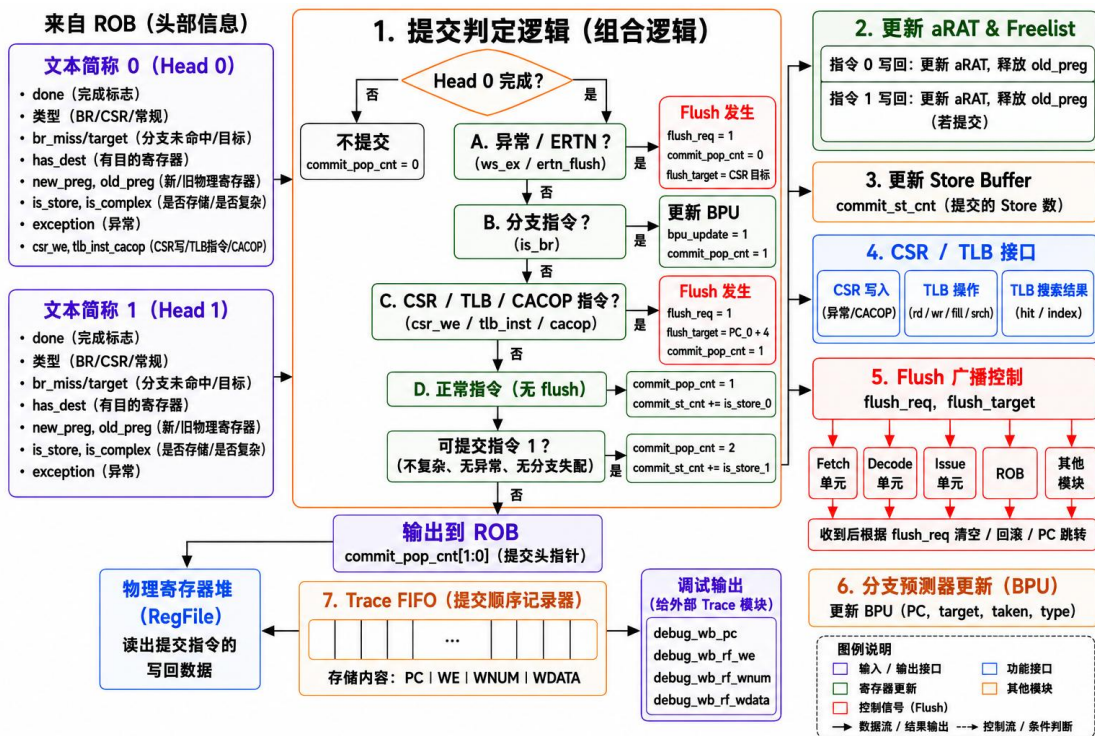
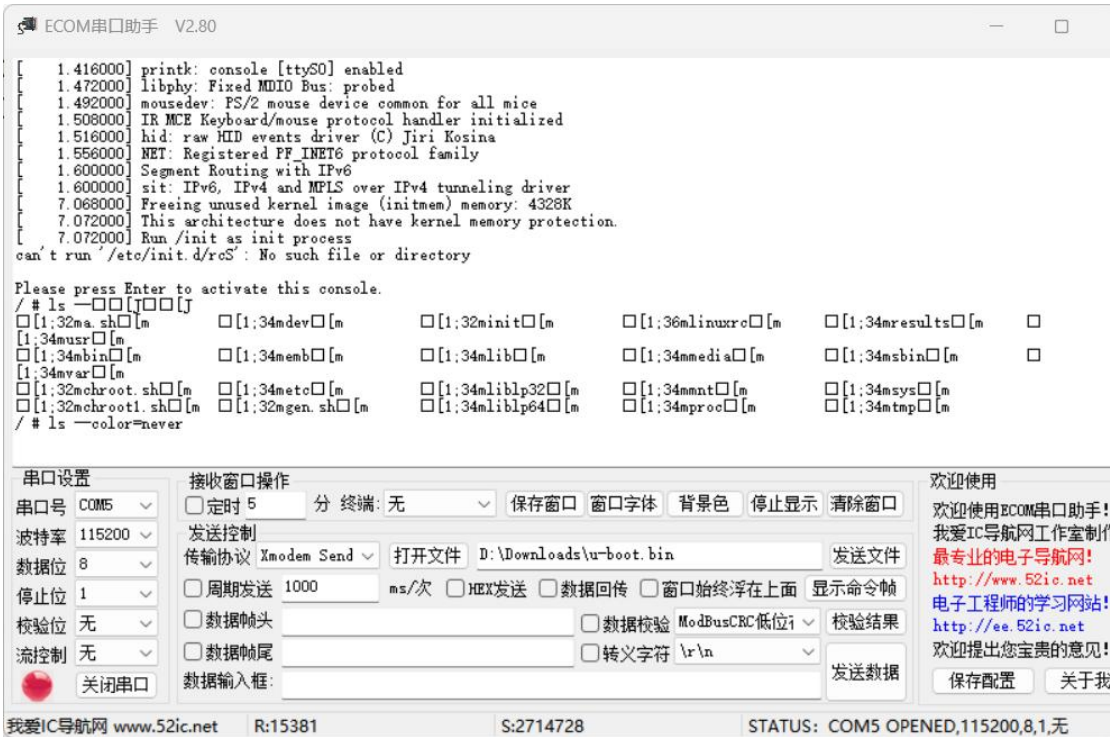


图 3- 2 提交级设计

第四章 结果展示与参考设计说明

4.1 启动 Linux 系统

本处理器可通过 u-boot 网络加载并运行内核，使用的内核镜像为龙芯开源项目 la32r-Linux 中发行版的 vmlinux 镜像[6]，具体操作可见 Chiplab 中的介绍[2]。



4.2 参考设计说明

1. 处理器微架构设计参考 CPU 设计实战与超标量处理器设计。^[3-4]
2. 乘除法器参考 2024 年西北工业大学一队 gitee 开源仓库 (NSCSCC-2024-team: 2024 龙芯杯西工大一队日常仓库)
3. 二级 TLB 参考 2025 年西北工业大学四队 gitee 开源仓库 (NSCSCC-NPUTeam: 西工大龙芯杯团队赛总仓库 - Gitee.com)
4. AXI 转接桥和 Cache 设计参考龙芯开源项目 openLA500 的设计^[5]。

参考文献

- [1] 龙芯中科.龙芯架构 32 位精简版参考手册 V1.03[EB/OL].[2026-08-05]
- [2] 龙芯教育.chiplab[EB/OL].[2026-08-05]
<https://gitee.com/loongson-edu/chiplab>
- [3] 姚永斌.超标量处理器设计[M].北京:清华大学出版社,2014.
- [4] 汪文祥,邢金章.CPU 设计实战[M].北京:机械工业出版社,2024.
- [5] 龙芯教育.openLA500[EB/OL].[2026-08-05].
<https://gitee.com/loongson-edu/open-la500>
- [6] 龙芯教育.la32r-Linux[EB/OL].[2026-08-05].
<https://gitee.com/loongson-edu/la32r-Linux>